

MÜHENDİSLİK FAKÜLTESİ

YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

PROGRAMLAMA -2 PROJE RAPORU

Proje Adı: Platform Koşucu (2D Platformer Game)

Geliştiren: Bengüsu Levent-250229050

1. Giriş ve Proje Özeti

Bu projede, C++ programlama dili ve SFML (Simple and Fast Multimedia Library) kütüphanesini kullanarak, nesne yönelimli programlama (OOP) prensiplerine uygun 2 boyutlu bir platform oyunu geliştirdim. Projeyi yaparken temel amacım; dinamik bir oyun döngüsü (game loop) kurmak, klavyeden gelen kullanıcı girdilerini anlık yakalamak, nesnelerin birbiriyle çarpışma algılamasını (collision detection) yönetmek ve ses/görsel gibi kaynakları performanslı bir şekilde ortaya koymaktır.

2. Kod Mimarisi ve Sınıf Yapısı

Projenin kod yapısını temiz teslim standartlarına uygun olarak tasarladım. Kod karmaşasını önlemek için kaynak kodlarımı `src/` klasöründe, oyunda kullandığım tüm materyalleri ise `assets/` klasöründe topladım.

- main.cpp (Oyun Yönetimi ve Döngüsü):** SFML'in `sf::RenderWindow` üzerinden oyun penceresini ve ana döngüyü kontrol ettim. Oyun ilk açıldığında kullanıcıyı estetik, pastel pembe tonlarında tasarlanmış "VESPERA" başlangıç ekranı karşılamaktadır. Oyuncu `R` tuşuna bastığı anda ana oyun döngüsü açılır ve oyun alanına geçiş yapılır. Oyun içi zaman yönetimini ve akışı `sf::Clock` kullanarak çözdüm.
- Player.cpp ve Player.hpp (Oyuncu Sınıfı):** Karakterin tüm fiziksel ve görsel özelliklerini bu sınıf içinde kapsülledim (encapsulation). Karakterin sağa/sola koşma ve platformlar arasında zıplama hareketlerini yerçekimi kuvvetini esas alarak uyguladım. Karakterin o anki hareket durumuna ve yönüne göre (texture) yüklenmesini sağladım.
- Kaynak Yönetimi (Asset Management):** Oyundaki arka plan görsellerini, karakter tasarımlarını, altın, can ve skor simgelerini projenin başında `assets/` klasöründen dinamik olarak belleğe yükledim. Böylece oyun akarken sürekli diskten okuma yapılmasını engelleyerek performansı artırdım.

3. Oyun Kontrolleri ve Kullanıcı Arayüzü Mekanizmaları

Oyuncunun oyun ile etkileşime girmesini sağlayan tüm girdi kontrollerini SFML'in `sf::Keyboard` kütüphanesi yardımıyla anlık olarak çalışacak şekilde kodladım. Oyundaki kontrol şeması ve arayüz şu şekildedir:

- Yatay Hareket:** Karakterin oyun haritasında sağa ve sola ilerlemesi için **A** ve **D** tuşları optimize edilmiştir.
- Dikey Hareket (Zıplama):** Platformlar arasında geçiş yapmayı ve engelleri aşmayı sağlayan zıplama hareketi **Space (Boşluk)** tuşu ile oynanır. Yerçekimi ivmesiyle ortak çalışan bu tuş, karakterin havadaki süzülme hareketini kontrol eder.
- Saldırı Mekanizması (Ateş Etme):** Karakterin karşısına çıkan engelleri veya hedefleri vurabilmesi için tasarlanan ateş etme/saldırı sistemi **F** tuşu ile aktif edilmektedir.
- Ekran Modu Yönetimi:** Kullanıcı deneyimini artırmak ve oyuncunun oyun atmosferine daha rahat odaklanabilmesini sağlamak amacıyla, pencere modları arasında dinamik geçiş sağlayan **ESC** tuşu aktif edilmiştir. Oyuncu bu tuşa bastığında oyun **Tam Ekran (Fullscreen)** moduna geçiş yapmaktadır.

4. Kullanılan Teknolojiler ve Kütüphaneler

- **C++17 Standardı:** Modern C++ özelliklerinden, akıllı işaretçilerden ve güvenli bellek yönetiminden yararlanmak adına projeyi C++17 standardında derledim.
- **SFML 3.0 (Simple and Fast Multimedia Library):** Projede kütüphanenin en güncel sürümü olan SFML 3.0 mimarisi tercih edilmiştir. Bu sayede modern grafik API'ları ve yenilenmiş ses/pencere yönetim yapıları entegre edilmiştir.
 - `sfml-graphics`: Karakterlerin, platform bloklarının ve yazıların ekrana çizilmesi için kullandım.
 - `sfml-window`: Pencere yönetimi, ekran modları ve klavye girdilerinin (`sf::Keyboard`) takibi için kullandım.
 - `sfml-audio`: Oyundaki zıplama, altın toplama, yanma ve kazanma gibi durumlar için `.wav` uzantılı ses efektlerini `sf::Sound` ile sisteme entegre ettim.

5. Projenin Derlenmesi ve Platform Bağımsızlığı (Cross-Platform)

Proje, mimari olarak tamamen **cross-platform** (platform bağımsız) bir yapıda geliştirilmiştir. Yazılan kaynak kodları ve SFML 3.0 mantığı, **Windows** ve **macOS** işletim sistemlerinde hiçbir kod değişikliği gerektirmeden çalışabilecek şekildedir. Sadece derleme aşamasında sistemlerin kütüphane yolları değişiklik göstermektedir.

Projenin Unix tabanlı (macOS) sistemlerde sorunsuz ve kütüphane bağımlılıkları eksiksiz bir şekilde derlenebilmesi için terminal şu komut satırı üzerinden derledim:

```
g++ -std=c++17 src/main.cpp src/Player.cpp -o game -I/opt/homebrew/include  
-L/opt/homebrew/lib -lsfml-graphics -lsfml-window -lsfml-system -lsfml-  
audio && ./game
```

Bu komut; bağımlılıkları bağlayarak `game` isimli çalıştırılabilir dosyayı üretir ve oyunu otomatik olarak başlatır.

Proje bir kez derlendikten sonra, tekrar derleme yapılmasına gerek kalmadan doğrudan terminale aşağıdaki komut yazılarak da oyun sorunsuz bir şekilde çalıştırılabilmektedir:

```
./game
```

Ayrıca projenin ana dizinine eklenen **Makefile** sayesinde, sadece `make` komutu çalıştırılarak da tüm bu derleme süreçleri tamamen otomatikleştirilmiştir. Proje Windows ortamına taşındığında ise Visual Studio veya MinGW derleyicileri üzerinden ilgili SFML 3.0 `.lib/.dll` dosyaları bağlanarak doğrudan derlenebilmektedir.

6. Sonuç

Geliştirdiğim "Platform Koşucu" projesi sayesinde, sıfırdan bir oyun motorunun temel çalışma mantığını ve oyun döngüsünü deneyimleme fırsatı buldum. Teori olarak gördüğümüz Nesne Yönelimli Programlama (OOP) kavramlarını bu projede pratiğe döktüm. Geliştirme sürecinin başında SFML kütüphanesinin çalışma yapısına ve dokümantasyonuna tam anlamıyla aşina olmamam, özellikle harita tasarımında kullanılan **tile (kiremit/blok) yerleşim mantığı** uygulamasında teknik zorluklar yaşamama neden olmuştur. Proje büyüdükçe artan kod satırlarında hata arama (debugging) ve farklı sınıfları mimari açıdan birbirine entegre etme süreçleri benim için ciddi birer problem çözme pratiği olmuştur. Karşılaştığım bu zorlukları deneme-yanılma yöntemi sayesinde projeyi akademik teslim standartlarına uygun ve başarıyla çalışır bir şekilde tamamladım.